Platform Profit Maximization for Space-Air-Ground Integrated Computing Power Network Supplied by Green Energy

Xiaoyao Huang[†,‡], Remington R. Liu[†], Bo Lei[†], Wenjuan Xing[†], Xing Zhang[‡]
[†]Network Technology Institute, Research Institute China Telecom, China
[‡]School of Information and Communication Engineering, Beijing University of Posts and Telecommunications, China
Email: {huangxy32, leibo,xingwj}@chinatelecom.cn, remington.liu@outlook.com, hszhang@bupt.edu.cn

ICC 2024 - IEEE International Conference on Communications | 978-1-7281-9054-9/24/$31.00 ©2024 IEEE | DOI: 10.1109/ICC51166.2024.10622297

***Abstract*—The rapid expansion of computing needs from emerging applications pushes a large amount of deployment of computing infrastructures and corresponding energy cost and greenhouse gas emissions of computing generate great concern. In this paper, we study how to maximize the platform profit by optimizing task scheduling in the Space-Air-Ground integrated Computing Power Network supplied by green energy while considering both the user requirements and dynamics of green energy. First, we formalize the problem as a binary integer linear programming problem that is NP-hard. The problem is then further modeled as a Markov decision process. Considering the dual dynamics of user requests and the generation of green energy, we propose a task scheduling strategy based on deep reinforcement learning, which can predict power generation based on the current operating status of each hydroelectric power station and also provide a scheduling strategy. Extensive experiments demonstrate that the proposed algorithm performs better than the baseline algorithms.**

I. Introduction

With the bloom of artificial intelligence (AI) applications (e.g.augmented reality, objective recognition, video processing, etc. [1], [2], [3]) and development of industrial Internet of Things, traditional network technology with separating network and computing [4], [5] has become inefficient in meeting the requirements of computationally intensive and latency critical tasks for these applications due to the potential mismatch of resource allocation and also resulted in low utilization. In addition, training of AI model (e.g. large language model [6]) brings unprecedented computing demands. A promising technology to address the challenge mentioned above is the Space-Air-Ground Integrated Computing Power Network (SAGICPN) [7], [8], which integrates the communication network with heterogeneous computing resources including edge servers, cloud servers, and satellite servers to offer application-requirement-oriented task scheduling. This integration is not limited to terrestrial networks, but extends to space and air networks, providing a comprehensive solution for the efficient allocation of computing resources.

However, enormous energy consumption is consumed and huge amount of carbon emission is released due to the massive computing of diversified tasks. Electricity cost places great pressure on computing power service providers and increases the price of service for users. Additionally, an increase in carbon emission can be damaging to the environment. Taking these issues into consideration, hydropower is an effective solution that can be introduced to SAGICPN [9].

As a supplementary energy source to commercial thermal power, hydropower is green and low cost. However, it is also characterized by unstable power generation and deployment location dependent on the distribution of natural resources [10]. In addition, the power generated by hydropower will be dropped without being used because of the lack of practical storage medium. The system platform works to schedule the tasks to the servers whose profit equals the income of task processing minus the energy cost and time occupation cost.

As a result, it is of great significance to maximize the platform profit for the SAGICPN equipped with hydropower supply by optimizing the task scheduling and resource allocation considering the effective usage of hydropower.

In this paper, we study how to maximize the platform profit by optimizing task scheduling in the SAGICPN supplied by hydropower while considering the dynamic characteristics of both the user tasks and the generation of hydropower with task requirements satisfied. First, we formalize the problem as a binary integer linear programming problem that is NP-hard. The problem is then further modeled as a Markov decision process.

Considering the above dual dynamics, we propose a DRL-based Task Scheduling strategy for the SAGICPN Supplied by green energy to learn the corresponding patterns and achieve optimized scheduling decisions. Extensive simulations are conducted and the results show the high performance of the proposed algorithm.

II. Related Work

Task scheduling plays an important role in CPN for supporting efficient resource utilization and improved quality of service satisfaction at user side. In this section, we present a brief review of existing work related to task offloading in computing power networks equipped with energy harvesting devices.

Ref. [11] investigates traffic offloading in heterogeneous small-cell networks, small cells are powered in a hybrid way including both the conventional commercial power supply and renewable energy harvested from the environment. The authors propose a joint optimization of traffic scheduling and

power allocation to minimize the total power consumption of macro and small cells on the grid, while ensuring the traffic requirement of each mobile user served. In Ref. [12], the authors propose a multi-relay-assisted computation offloading framework for multi-access edge computing systems with energy harvesting and develop a low-complexity online algorithm to select the optimal execution strategy for each task to minimize the average execution cost. The authors in Ref. [13] investigate computation offloading in fog computing system with energy harvesting technique. They formulate the optimization problem as a partially observable decentralized Markov decision process and use Lagrangian approach and the policy gradient method to find the optimal solution for the proposed problem. Ref. [14] presents a neighbor-aware distributed task offload algorithm in which the Internet of Things device takes its energy status and neighbor devices' decisions into account for the decisions on whether to process the task by itself and on which edge cloud is selected to offload the task.

Although there are many research works on task offloading in energy-harvest environments, the source of green energy in their systems is often solar energy or other relatively stable green energy sources. Hydropower generation, which has the characteristics of short-term randomness and long-term periodicity, is rarely considered. Short-term randomness pertains to the unpredictability of water flow, meaning that the flow of water will vary over time, whereas long-term periodicity relates to the daily pattern of water flow. This variability in power generation poses a challenge. Therefore, this paper studies task offloading in a SAGICPN with hydropower-enabled terrestrial networks and solar-powered satellites. By making good use of the short-term and long-term characteristics of hydropower, we can maximize the use of green energy and maximize the efficiency of the system.

III. System Model

In this section, we first describe the architecture of the SAGICPN supplied with the hydropower under study and then introduce the related delay, energy model, and profit model. Finally, we formulate the problem under study.

A. Network Model

Fig. 1 shows the SAGICPN system under study in this paper, which consists of N edge servers $\mathcal{N} = \{1, \cdots, N\}$, K cloud servers $\mathcal{K} = \{1, \cdots, K\}$ and M satellite servers $\mathcal{M} = \{1, \cdots, M\}$ connected to the network. Due to geographical location restrictions, the partial edge servers denoted by $\mathcal{G} = \{1, \cdots, G\}, \mathcal{G} \subset \mathcal{N}$ are powered by hydropower. The satellite is equipped with solar panels, making it powered by renewable energy. When a user sends a computing task to the system, the platform of the SAGICPN system will assign the task to the most suitable servers to maximize the total platform profit considering the user requirement and the generation of hydropower. Let the set of tasks be denoted by $\mathcal{I}$. Task $T_i, i \in \mathcal{I}$ can be represented by a quadruple $T_i = \langle l_i, r_i, t_i^s, t_i^d \rangle$ where l_i is the data size of the task, r_i is the CPU round

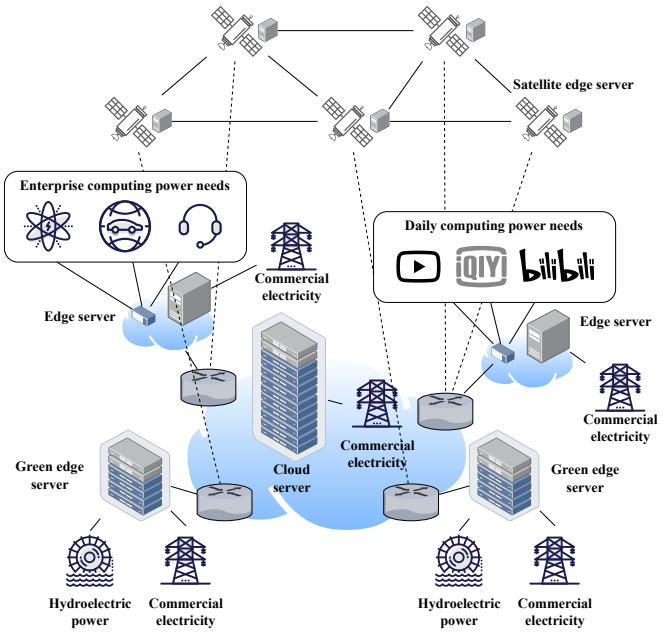


Fig. 1: Architecture of the SAGICPN supplied with hydropower under study

required for one bit, t_i^s is the start time of the task and t_i^d is the delay deadline of the task. Assume there are no dependencies between tasks.

For the reason that the profit of the platform is mainly determined by the total processed workload of all tasks minus the energy cost and time occupation cost, it is important to optimize the task scheduling by achieve a good matching of tasks and servers to make full use of generated hydropower while satisfying task requirements.

B. Computing Model

Once tasks are received by the server, they will be processed in the order they were received, following a first-come, first-served approach. The computing power of the server $k \in \mathcal{N} \cup \mathcal{K} \cup \mathcal{M}$ is f_k and the computing delay $t_{k,i}^p$ of task T_i can be calculated as

$$t_{k,i}^p = \frac{l_i r_i}{f_k}. \tag{1}$$

Since each server k has a service queue with a certain workload $L_{k,i}^q$ before task T_i arrived, the queuing delay of task T_i is

$$t_{k,i}^q = \frac{L_{k,i}^q}{f_k}. \tag{2}$$

C. Delay Model

A task can be offloaded by ground computing nodes (e.g. edge servers, cloud servers) or satellites, which will result in different transmission delays.

For ground computing nodes, the transmission delay $t_{k,i}^t$ is computed as:

$$t_{k,i}^t = \sum_{l \in LinkS_{k,i}} \frac{l_i}{v_l^g} + \frac{Distance_{k,i}}{V}, \tag{3}$$

where the first term is the transmission delay, with v_l^g as the link bandwidth of the corresponding network link belonging to the total link set $LinkS_{k,i}$ of the route from user k to server s. The second term represents the propagation delay, with $Distance_{k,i}$ being the distance between the user k and the server i, and V being the speed of light. And for satellites, due to the high altitude of the satellite, the channel gain of the satellite is assumed to be a constant value [15]. When the bandwidth and transmit power are set well, the transmission rate is a constant value [16]. Therefore, the transmission delay $t_{k,i}^t$ is

$$t_{k,i}^t = \frac{l_i}{v_l^s}, \tag{4}$$

where v_l^s is the data rate of the device-satellite link.

The delay of a task includes communication delay, queuing delay, and computing delay. Therefore, the total delay of task T_i offloading to server k is

$$t_{k,i} = t_{k,i}^t + t_{k,i}^q + t_{k,i}^p. \tag{5}$$

D. Energy Model

1) Energy Generation: In this paper, we focus on a green energy environment enabled by hydropower. Hydropower stations are geographically distributed and connected to the nearest edge server. The edge server will receive energy from the hydroelectric power station, but the data centers, which have a great deal of computing power and are responsible for other tasks, will not be directly supplied with energy from the hydroelectric power station due to their high energy consumption.

Assume that the power generation capacity of the hydroelectric station $n \in \mathcal{N}$ at time t is $c_n(t)$. Therefore, $c_n(t)$ can represent a time series, used to represent the power generation capacity at different times t. Since $c_n(t)$ is a discrete function, the continuous quantity $\tilde{c}_n(t)$ can be obtained by interpolating $c_n(t)$. Applying integration to $\tilde{c}_n(t)$, we can get the power generation in time periods t_1 and t_2. Therefore, the power generation $e_n^g\{t_1, t_2\}$ in the time periods t_1 and t_2 of the hydroelectric station n can be calculated by

$$e_n^g\{t_1, t_2\} = \int_{t_1}^{t_2} \tilde{c}_n(t)\mathrm{d}t. \tag{6}$$

2) Energy Consumption: The energy cost of a server $k \in \mathcal{N} \cup \mathcal{K} \cup \mathcal{M}$ for processing a task T_i can be calculate by

$$e_{k,i}^c = \beta_e l_i f_k^2, \tag{7}$$

where β_e is the energy consumption coefficient.

For server k in the set $\mathcal{N}$, hydroelectric power stations can provide some of the energy needs, thus decreasing the amount of commercial electricity used. So $e_{k,i}^c$ can be reformed as

$$e_{k,i}^c = \left[\beta_e l_i f_k^2 - e_n^g\{t_i^s, t_i^s + t_{k,i}\}\right]^+, \tag{8}$$

where $[x]^+ = \max\{x, 0\}$.

For server k in the set $\mathcal{M}$, solar panels power the satellite with power P_s. So $e_{k,i}^c$ can be reformed as

$$e_{k,i}^c = \left[\beta_e l_i f_k^2 - P_s t_{k,i}^p\right]^+, \tag{9}$$

To sum up, the energy cost of a server for processing a task T_i can be calculate by

$$e_{k,i}^c = \begin{cases} \beta_e l_i f_k^2, \ k \in \mathcal{K} \\ \left[\beta_e l_i f_k^2 - e_n^g\{t_i^s, t_i^s + t_{k,i}\}\right]^+, \ k \in \mathcal{N} \\ \left[\beta_e l_i f_k^2 - P_s t_{k,i}^p\right]^+, \ k \in \mathcal{M} \end{cases} \tag{10}$$

E. Profit Model

The binary variable $x_{k,i}$ can take on either of two values: 0 or 1. If $x_{k,i} = 0$, then the task T_i is not offloaded to the server k. Conversely, if $x_{k,i} = 1$, then the task T_i is offloaded to the server k. A task can be offloaded or not, and can be offloaded by at most one server. Let $\mathcal{H} = \mathcal{N} \cup \mathcal{K} \cup \mathcal{M} \cup \{\text{null}\}$, we have

$$\sum_{k \in \mathcal{H}} x_{k,i} = 1, \forall i \in \mathcal{I}, \tag{11}$$

where $x_{\text{null},i} = 1$ means not offload the task T_i to any server.

The price of processing one-bit data of the task on the server is ρ, so the total income is $I = \sum_{k \in \mathcal{N} \cup \mathcal{K} \cup \mathcal{M}} \sum_{i \in \mathcal{I}} \rho x_{k,i} l_i$. In addition, the cost of a server k for processing a task T_i consists of the energy cost and the time occupation cost which can be calculated as $C = \sum_{k \in \mathcal{N} \cup \mathcal{K} \cup \mathcal{M}} \sum_{i \in \mathcal{I}} x_{k,i} \left(w_e e_{k,i}^c + w_t \frac{l_i r_i}{f_k}\right)$. Therefore, the profit can be expressed as

$$\begin{aligned} P &= I - C \\ &= \sum_{k \in \mathcal{N} \cup \mathcal{K} \cup \mathcal{M}} \sum_{i \in \mathcal{I}} x_{k,i} \left(\rho l_i - w_e e_{k,i}^c - w_t \frac{l_i r_i}{f_k}\right). \end{aligned} \tag{12}$$

F. Problem Formulation

In this paper, the objective is to maximize the total platform profit, which equals the total income for task processing processed workload minus the total processing cost, while satisfying the delay requirements of such tasks. Accordingly, the CPN platform profit maximization problem is formulated as follows.

$$\textbf{P1:} \max_{\boldsymbol{x}} \sum_{k \in \mathcal{N} \cup \mathcal{K} \cup \mathcal{M}} \sum_{i \in \mathcal{I}} x_{k,i} \left(\rho l_i - w_e e_{k,i}^c - \frac{l_i r_i}{f_k}\right) \tag{13}$$

$$\text{s.t.} \sum_{k \in \mathcal{H}} x_{k,i} = 1, \forall i \in \mathcal{I}, \tag{13a}$$

$$\sum_{k \in \mathcal{N} \cup \mathcal{K} \cup \mathcal{M}} x_{k,i} t_{k,i} \leqslant t_i^d, \forall i \in \mathcal{I}, \tag{13b}$$

$$x_{k,i} = \{0, 1\}, \forall k \in \mathcal{H}, \forall i \in \mathcal{I}, \tag{13c}$$

The constraint (13a) indicates that a task can only be offloaded to one server at most. The constraint (13b) means that the task can only be offloaded to a server that can complete the task within the time limit, otherwise the task will not be offloaded. The constraint (13c) indicates that the decision variable $x_{k,i}$

is a binary variable. **P1** is a typical Binary Integer Linear Programming (BILP) problem, which is difficult to solve due to its NP-hard nature. Considering the dynamic characteristics of both the user tasks and the generation of hydropower, we proposed a DRL-based Task Scheduling strategy for the Space-Air-Ground Integrated Computing Power Network Supplied by Green energy (DTSG) to solve this problem effectively depending no future information.

IV. Proposed Algorithm

In order to be able to use deep reinforcement learning to solve **P1**, we need to first formulate the problem as an Markov Decision Process (MDP). In this section, we first model the MDP of the profit maximization problem, and then introduce the deep reinforcement learning model of the proposed algorithm. Finally, the detailed procedure of the DRL-based task scheduling strategy is presented.

A. Modeling of Markov Decision Process

To apply deep reinforcement learning to the task scheduling in hydropower generation environment, we first formulate our problem as a Markov Decision Process (MDP), which can be expressed as a 5-tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma \rangle$, where $\mathcal{S}$ is the state set, $\mathcal{A}$ is the action set, $\mathcal{R}$ is the reward function, $\mathcal{P}$ is the state transition function, γ is the discount rate, $\gamma \in (0, 1]$. The MDP is modeled as follows.

1) State ($\mathcal{S}$): The state space should include the state of the task and the servers, so the state set $\mathcal{S}$ is defined as $\mathcal{S} = \left\langle \boldsymbol{L}_i^q, r_i, l_i, t_i^s, t_i^d \right\rangle$, where $\boldsymbol{L}_i^q$ is a vector of length $|\mathcal{H}|$ containing the $L_{k,i}^q$ values.
2) Action ($\mathcal{A}$): Whenever a task arrives, the task must be scheduled to a certain server or not. Therefore, the action space is define as $\mathcal{A} = \langle \boldsymbol{x}_i \rangle$, where $\boldsymbol{x}_i$ is a vector of length $|\mathcal{H}|$ containing the $x_{k,i}$ values.
3) Reward ($\mathcal{R}(\cdot, \cdot)$): The reward function should reflect the optimization goal, so the objective function of (13) is used as the reward function. So $\mathcal{R}(s_i, a_i) = \sum_{k \in \mathcal{N} \cup \mathcal{K}} \sum_{i \in \mathcal{I}} x_{k,i} \left(\rho l_i - w_e e_{k,i}^c - \frac{l_i r_i}{f_k} \right)$.

B. Proposed DTSG Strategy

In this article, we proposed a DRL-based Task Scheduling strategy for the space-air-ground integrated computing power network supplied by green energy. The framework of DRL is Proximal Policy Optimization (PPO) [17] with Clip function.

The DTSG based on PPO follows a learning process that begins when a task is sent to the task scheduler. The DRL observes the task's status and the status of the servers, as described in the MDP model. PPO then makes a decision based on the current policy, assigning the task to a specific server. Through continuous interaction with the environment, PPO gradually learns the most suitable policy to schedule tasks in any state, aiming to maximize the reward function and the platform's benefits. The utilization of hydropower is included in the interactive learning process of PPO. The following section will explain how PPO updates policies through continuous interaction.

In PPO framework, the advantage function is calculated as follows.

$$A_\pi(s_t, a_t) = Q_\pi(s_t, a_t) - V_\pi(s_t), \tag{14}$$

where $Q_\pi(s_t, a_t) = E_{s_{t+1}, a_{t+1}, \cdots} \left[\sum_{l=0}^{\infty} \gamma^t r_{t+l} \right]$ represents the expected profit of executing the offloading decision a_t when the system and also the task are in the state of s_t and $V_\pi(s_t) = E_{a_t, s_{t+1}, \cdots} \left[\sum_{l=0}^{\infty} \gamma^t r_{t+l} \right]$ represents the expected profit of the state of s_t. In addition, γ^t represents the reward discount.

Taking into account the effect of bias and variance, RMS utilizes the generalized advantage estimation (GAE) to estimate the advantage function, which is calculated in the following way.

$$A_t = \sum_{l=0}^{\infty} (\gamma\phi)^l \kappa_{t+l}^V, \tag{15}$$

where ϕ is used to balance bias and variance, and κ_{t+l}^V can be calculated by $\kappa_t^V = r_t + \gamma V_{\pi'}(s_{t+1}) - V_{\pi'}(s_t)$, where r_t represents the profit of the system for the decision made at time t. The importance sampling weight is calculated as follows.

$$\eta_\theta = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta'}(a_t|s_t)}, \tag{16}$$

where $\pi_\theta(a_t|s_t)$ is the updated policy function and $\pi_{\theta'}(a_t|s_t)$ is the pre-updated.

Then the loss function of PPO is defined as $J_{ppo}(\theta) = E\left[\min\left(\eta_\theta A_{\pi'}(s_t, a_t), \text{clip}\left(\eta_\theta, 1-\delta, 1+\delta\right) A_{\pi'}(s_t, a_t) \right) \right]$, where clip() limits η_θ to vary between $1-\delta$ and $1+\delta$.

The procedure for DTSG is shown in **Algorithm 1**.

V. Performance Evaluation

A. Simulation Setup

In the simulation, the main parameter values are shown in Table I. The proposed algorithm is compared with the following benchmark algorithms:

- **Greedy Scheduling for Profit Maximization (Greedy):** The Greedy algorithm estimates the cost of every server and then assigns each task to the server leading to the maximum profit provided that they meet the delay.
- **Random Scheduling (Random):** The Random algorithm randomly assigns each task to the server that meets the delay constraints.

B. Convergence

We assess the convergence of DTSG in a simulated environment, as depicted in Fig. 2. The training process was conducted with a total of 5000 episodes and a learning rate of 0.001, with a task arrival rate of 20. The simulation environment was built on the Gym framework [18]. It is evident from Fig. 2 that DTSG converges to a certain value after 2000 episodes. The converged DTSG is then used for further experiments and comparisons.

Algorithm 1: Procedure for DTSG

Input: Learning rate, β_s, $Distance_{k,s}$, v_l, γ^t, $|LinkS|$, ρ.
Output: Task scheduling decisions.

Randomly initialize network parameters, initialize policy π_θ.
for *episode = 0,1,2,...* **do**
  for t = *0,1,2,...,T* **do**
    Observe the state: $\langle s_t \rangle = \langle \boldsymbol{L}_{i,t}^q, r_i, l_i, t_i^s, t_i^d \rangle$;
    Get a_t based on policy π_θ;
    Execute a_t to offload task i to a server k;
    Get new state s_{t+1};
    Obtain the reward $R(s_t, a_t)$;
    Store $\langle s_t, a_t, R(s_t, a_t), s_{t+1} \rangle$ in set of interaction D for training;
    Compute advantage estimation A_t according to Eq. (15);
  end
  Update offloading policy by using clipped loss function $\theta = \arg\max_\theta \frac{1}{|D|T} \sum_{\epsilon \in D} \sum_{t=0}^{T} J_{ppo}(\theta)$;
  Update profit estimation network by using mean-squared error function $\phi = \arg\min_\theta \frac{1}{|D|T} \sum_{\epsilon \in D} \sum_{t=0}^{T} \left(V_\phi(s_t) - \hat{R} \right)^2$;
end

TABLE I: Parameters value

Parm	Value	Unit	Parm	Value	Unit
$\lvert\mathcal{K}\rvert$	1	-	l_i	[5, 50]	Mb
$\lvert\mathcal{N}\rvert$	4	-	r_i	[120, 900]	Round/bit
$\lvert\mathcal{M}\rvert$	2	-	t_i^d	[2, 10]	second
f_k	[200, 900]	GHz	ρ	5	-
w_e	0.0012	-	w_t	100	-
γ	0.99	-	ϕ	0.99	-
δ	0.2	-	$Distance_{k,i}$	[30, 720]	km
V	3×10^8	m/s	$\lvert LinkS_{k,i}\rvert$	[1, 24]	-
λ	[10, 550]	-	v_l^g	[100, 500]	Mb/s
v_l^s	20	Mb/s	t_i^s	[0, 24)	hour

C. Platform Profit

We assess the performance of DTSG in platform profit by varying the task arrival rate from 10 to 500 and comparing it with the baseline algorithms. The results are shown in Fig. 3(a). It can be seen from Fig. 3(a) that no matter in loose task flow or tight task flow, the performance of DTSG is better than that of the baseline algorithms. Furthermore, in the case of loose task flow, the greedy algorithm performs better than the random algorithm, while in the case of tight task flow, the random algorithm performs better. This is because the random algorithm can balance the load of each server in the system, and balance plays a dominant role in system performance in tight task flow. On average, DTSG has a 21.77% improvement over the Random algorithm and a 16.34% improvement over the Greedy algorithm.

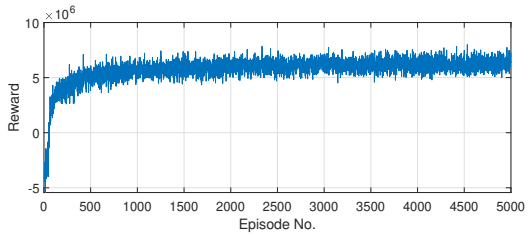


Fig. 2: Learning Curve

D. Offloading Rate

We assess the performance of DTSG in the offloading rate under the same conditions. The results are shown in Fig. 3(b). It can be seen from Fig. 3(b) that no matter in loose task flow or tight task flow, the performance of DTSG is better than that of the baseline algorithms. On average, DTSG has a 38.21% improvement over the Random algorithm and a 25.23% improvement over the Greedy algorithm.

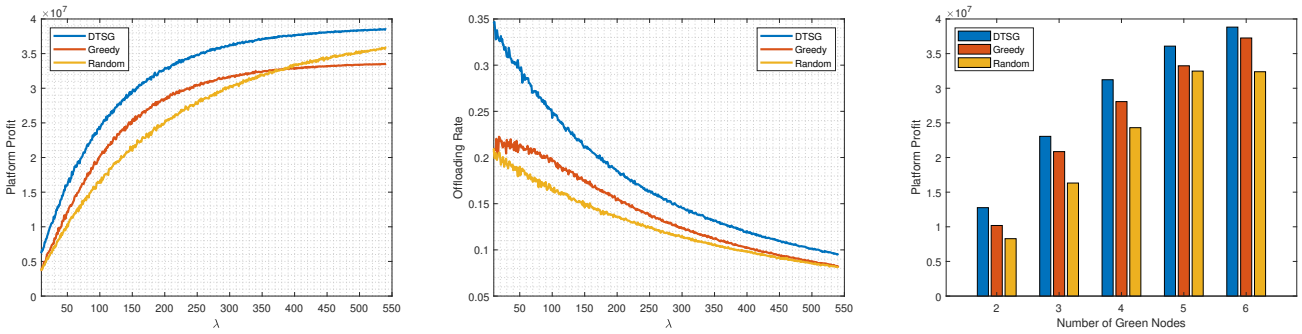
E. Varying Green Nodes

We altered the amount of green nodes in the network from 2 to 6, while the amount of non-green nodes stayed at 2. We then compared the performance changes of each algorithm under the various green node conditions. To begin, we compared the platform profit of each algorithm under the different green node conditions. The experimental results are shown in Fig. 3(c). The results of Fig. 3(c) demonstrate that the proposed algorithm outperforms the Random and the Greedy algorithms in terms of system income when the number of green nodes in the network is varied. As the number of green nodes increases, the system benefit also increase. This is due to the fact that a larger number of green nodes allows for more green energy to be utilized, thus reducing operating costs. On average, DTSG has a 21.15% improvement over the Random algorithm and a 8.21% improvement over the Greedy algorithm.

In addition, we also compared the offloading rate under different green nodes. The experimental results are shown in Fig. 4. As can be seen from Fig. 4, the offloading rate of the proposed algorithm is higher than that of the Greedy algorithm and the Random algorithm. And as the number of green nodes increases, the offloading rate gradually increases. This is because the increase in computing resources in the network allows more tasks to be offloaded. On average, DTSG has a 25.56% improvement over the Random algorithm and a 19.12% improvement over the Greedy algorithm.

VI. Conclusion

In this paper, we studied how to maximize the platform profit by optimizing task scheduling in the computing power network supplied by hydropower while considering the dynamic characteristics of both the user tasks and the generation of hydropower with task requirements satisfied. Firstly, we formalized the problem as a binary integer linear programming problem that is NP-hard. The problem was then further



(a) (b) (c)

Fig. 3: Experimental results. (a) Platform profit in different task arrival rate, (b) offloading rate in different task arrival rate, and (c) system benefit under different green nodes.

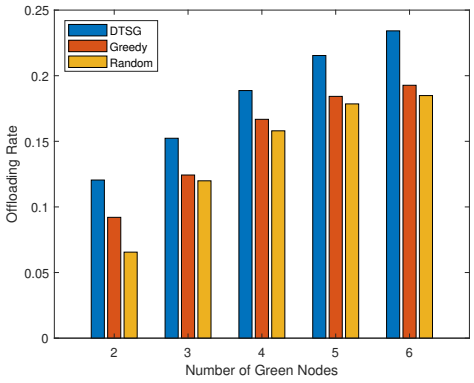


Fig. 4: Offloading Rate under Different Green Nodes

modeled as a Markov decision process. Considering the above dual dynamics, we proposed a DRL-based Task Scheduling strategy for the Space-Air-Ground Integrated Computing Power Network Supplied by Green energy (DTSG) to learn the corresponding patterns and achieve optimized scheduling decisions. Extensive simulations were finally conducted and the results showed the high performance of the proposed algorithm.

ACKNOWLEDGMENT

This work was supported by National Key Research and Development Program: 2022YFB2901400.

REFERENCES

[1] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1738–1762, 2019.

[2] X. Jiang, F. R. Yu, T. Song, and V. C. M. Leung, "A survey on multi-access edge computing applied to video streaming: Some research issues and challenges," *IEEE Communications Surveys & Tutorials*, vol. 23, no. 2, pp. 871–903, 2021.

[3] T. Rodrigues, K. Suto, and N. Kato, "Edge cloud server deployment with transmission power control through machine learning for 6G Internet of things," *IEEE Transactions on Emerging Topics in Computing*, vol. 9, no. 4, pp. 2099–2108, 2021.

[4] P. Mell and T. Grance, "The NIST definition of cloud computing." *Communications of the ACM*, vol. 53, no. 6, pp. 50–50, 2010.

[5] B. Kar, W. Yahya, Y.-D. Lin, and A. Ali, "Offloading using traditional optimization and machine learning in federated cloud–edge–fog systems: A survey," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 2, pp. 1199–1226, 2023.

[6] L. Floridi and M. Chiriatti, "Gpt-3: Its nature, scope, limits, and consequences," *Minds and Machines*, vol. 30, pp. 681–694, 2020.

[7] X. Tang, C. Cao, Y. Wang, S. Zhang, Y. Liu, M. Li, and T. He, "Computing power network: The architecture of convergence of computing and networking towards 6G requirement," *China Communications*, vol. 18, no. 2, pp. 175–185, 2021.

[8] Y. Sun, J. Liu, H. Huang, X. Zhang, B. Lei, J. Peng, and W. Wang, "Computing power network: A survey," *arXiv preprint arXiv:2210.06080*, 2022.

[9] L. Gu, W. Zhang, Z. Wang, D. Zeng, and H. Jin, "Service management and energy scheduling toward low-carbon edge computing," *IEEE Transactions on Sustainable Computing*, vol. 8, no. 1, pp. 109–119, 2022.

[10] J. Barzola-Monteses, J. Gomez-Romero, M. Espinoza-Andaluz, and W. Fajardo, "Hydropower production prediction using artificial neural networks: an ecuadorian application case," *Neural Computing and Applications*, pp. 1–14, 2022.

[11] Y. Wu, X. Yang, L. P. Qian, H. Zhou, X. Shen, and M. K. Awad, "Optimal dual-connectivity traffic offloading in energy-harvesting small-cell networks," *IEEE Transactions on Green Communications and Networking*, vol. 2, no. 4, pp. 1041–1058, 2018.

[12] M. Li, X. Zhou, T. Qiu, Q. Zhao, and K. Li, "Multi-relay assisted computation offloading for multi-access edge computing systems with energy harvesting," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 10, pp. 10 941–10 956, 2021.

[13] Q. Tang, R. Xie, F. R. Yu, T. Huang, and Y. Liu, "Decentralized computation offloading in iot fog computing system with energy harvesting: A dec-pomdp approach," *IEEE Internet of Things Journal*, vol. 7, no. 6, pp. 4898–4911, 2020.

[14] J. Lee and H. Ko, "Neighbor-aware distributed task offloading algorithm in energy-harvesting internet of things," *IEEE Internet of Things Journal*, vol. 10, no. 10, pp. 8744–8753, 2023.

[15] Y. Liu, L. Jiang, Q. Qi, and S. Xie, "Energy-efficient space–air–ground integrated edge computing for internet of remote things: A federated drl approach," *IEEE Internet of Things Journal*, vol. 10, no. 6, pp. 4845–4856, 2022.

[16] H. Liao, Z. Zhou, X. Zhao, and Y. Wang, "Learning-based queue-aware task offloading and resource allocation for space–air–ground-integrated power iot," *IEEE Internet of Things Journal*, vol. 8, no. 7, pp. 5250–5263, 2021.

[17] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[18] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "Openai gym," 2016.